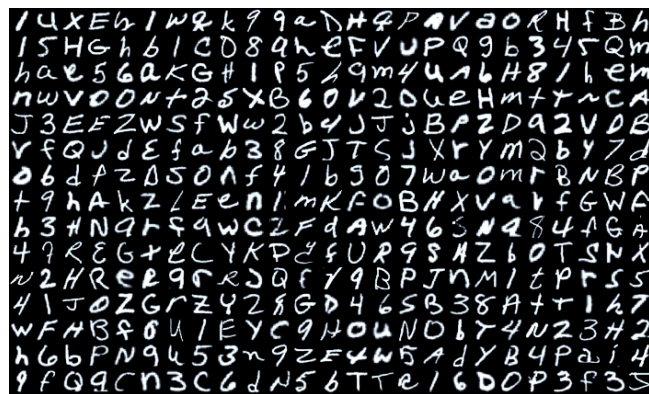


PROJET 4 EN MATHÉMATIQUES APPLIQUÉES - LEPL1507
GROUPE 03
RAPPORT FINAL

Classification robuste sous attaques adversariales



Étudiant·e·s :

ANTONUTTI Victor
DESMETTE Mateo
DJOUKOUO Marie Pascale
HERTOGHE Adèle
LEPOUTRE Florian
MBENGUE Abdou

Enseignant·e·s :

MASSART Estelle
KRINGS Gauthier
HENDRICKX Julien

Table des matières

1	Contexte du projet	1
2	Formulation du problème	1
3	Premier classifieur	1
3.1	Algorithme CNN (Convolutional Neural Network)	1
3.2	Principaux éléments d'un CNN	2
3.3	Choix d'implémentations	3
3.4	Entraînement	4
3.4.1	Fonction de perte	4
3.4.2	Optimiseur et Scheduler	4
3.5	Résultats et analyse	5
3.6	Autres pistes envisagées	7
4	Attaquants	7
4.1	Description du modèle d'attaque : perturbation ℓ_2	7
4.2	Objectif de l'attaque	7
4.3	Principe de l'algorithme	8
4.4	Stratégie utilisée	8
4.5	Hyperparamètres utilisés	9
4.6	Résultats et analyse de l'attaquant PGD	9
4.7	Second attaquant envisagé : Deepfool	10
4.7.1	Principe	10
4.7.2	Explication de notre implémentation	10
4.7.3	Hyperparamètres utilisés	11
4.7.4	Résultats et analyse	11
4.8	Comparaison PGD vs Deepfool	12
5	Deuxième classifieur	12
5.1	Description des améliorations techniques	12
5.1.1	Adversarial training	12
5.1.2	Evolution de l'architecture	13
5.2	Hyperparamètres finaux du classifieur	16
5.3	Résultats et analyse	16
5.4	Étude comparative des classifieurs de S4 et de S11	18
6	Outils de calculs utilisés	19
7	Ajouts supplémentaires au projet	19
7.1	Programme de dessin interactif	19
7.2	Recherche du plus petit ε adversarial	19
8	Conclusion	20
9	Utilisation de l'IA	20
A	Architecture du premier réseau de neurones	22
B	Entraînement	23

C	Attaque PGD pour le second classifieur	23
C.1	Attaquant utilisé : Projected Gradient Descent	23
C.1.1	Explication	23
C.1.2	Opérations fondamentales	24
C.1.3	Calendrier de pas cosinus	24
C.1.4	Règle de mise à jour PGD	24
C.2	Algorithme	25
D	Exemples d'attaques Deepfool	25
E	Autres pistes envisagées	26
F	Exemples d'utilisation du programme de dessin interactif	26
F.1	Programme avec epsilon fixé	27
F.2	Programme permettant de trouver le plus petit epsilon fonctionnel	28

1 Contexte du projet

L'objectif du Hackathon Cybersécurité IA 2026 est clair, nous avons été recrutés pour concevoir le modèle de reconnaissance d'images calligraphiées le plus robuste possible, capable de résister à des attaques adversariales et maintenir une précision aigüe. En effet, le CISO de la Banque Européenne de Sécurité (BES) est confrontée à une menace discrète et dangereuse : des fraudeurs tentent de bruite les images de chèques afin de tromper les logiciels de lecture automatisée, fausser les informations bancaires et détourner des fonds. Les pertes financières sont estimées à plusieurs millions d'euros par an et il est temps d'agir.

Ce rapport reprend notre travail de conception d'un classifieur permettant de résister aux attaques des fraudeurs, depuis sa première version jusqu'à sa version optimisée en terme de robustesse.

2 Formulation du problème

On considère donc un problème de classification d'image. Soit

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N, \quad x_i \in [0, 1]^d, \quad y_i \in \{0, \dots, C-1\},$$

où $d = 28 \times 28 = 784$ est la dimension des images d'entrées (images en nuances de gris) et $C = 47$ est le nombre de classes de EMNIST **Balanced**, le dataset utilisé. Le but de notre classifieur F est d'assigner, sur base d'un x_i , une classe $\hat{y}_i = F(x_i)$ de telle sorte que, sur l'ensemble des x_i , l'erreur commise soit minimale.

Le dataset EMNIST **Balanced** est un jeu de données qui contient des images de chiffres et de lettres manuscrits, ainsi que leurs véritables labels : 112 800 images d'entraînement et 18 800 images de test, réparties en 47 classes équilibrées. Chaque image est un carré de 28x28 pixels en noir et blanc, où chaque pixel est représenté par un entier entre 0 (noir) et 255 (blanc). Dans notre code, nous importons ce dataset grâce à la librairie **TorchVision**, ce qui transforme les images en `torch.FloatTensor` de taille 28x28 avec des valeurs entre 0.0 (noir) et 1.0 (blanc).

3 Premier classifieur

3.1 Algorithme CNN (Convolutional Neural Network)

Ce modèle de classification d'images est conçu pour traiter des données ayant une structure en grille, et plus précisément des matrices de pixels. Son architecture repose principalement sur les couches de convolutions, dont l'objectif va être d'extraire les caractéristiques propres à chaque image (courbes, lignes, etc), et de les compresser de sorte à réduire leur taille pour ne conserver que l'information pertinente.

Entrée Image en noir et blanc (tenseur d'ordre 2 et de dimension $1 \times 28 \times 28$ (matrice)), avec chaque valeur comprise entre 0 et 1.

Sortie Probabilité sur les 47 classes de EMNIST **Balanced**

$$x^1 \rightarrow \boxed{w^1} \rightarrow x^2 \rightarrow \dots \rightarrow x^{L-1} \rightarrow \boxed{w^{L-1}} \rightarrow x^L \rightarrow \boxed{w^L} \rightarrow z \rightarrow p \quad (1)$$

L'entrée passe séquentiellement dans une série de processus. Chaque processus est appelé "couche" du modèle. Le schéma 1 reprend la structure du modèle. L'entrée est dénotée par l'entrée x^1 et sert d'entrée à la couche w^1 , dont la sortie est x^2 , et ainsi de suite.

Ce réseau de neurones $f_\theta : \mathbb{R}^{28 \times 28} \rightarrow \mathbb{R}^{47}$ cartographie une image vers un vecteur de probabilités, et la classe prédite est

$$\hat{y}(x) = \arg \max_{c=1, \dots, 47} [f_\theta(x)]_c.$$

3.2 Principaux éléments d'un CNN

Couches de convolution Ces couches sont l'essence même d'un réseau de neurones convolutif. Durant celles-ci, plusieurs filtres (appris pendant la phase d'entraînement) de tailles égales sont appliqués à l'image pour reconnaître des patterns tels que des courbes, des bords, etc.

Le principe mathématique derrière est la convolution. Soit une image (de 28 sur 28 pixels dans notre cas) et un filtre (de 3 sur 3 pixels dans notre cas). Le filtre va glisser sur toute la surface de l'image et pour chaque position, calculer le produit scalaire entre les pixels de l'images et ceux du filtre. Le résultat est donc une matrice de taille légèrement réduite (26 sur 26 pixels dans notre cas) qui contient tous les produits scalaires correspondants. En pratique, nous effectuons du zero-padding autour de l'image, en amont de la convolution, afin de conserver une taille de 28 sur 28.

Mathématiquement, étant donnée une image d'entrée $x \in \mathbb{R}^{28 \times 28}$ et un filtre $k \in \mathbb{R}^{m \times m}$, la convolution discrète est définie par :

$$(x * k)(i, j) = \sum_{u=1}^m \sum_{v=1}^m x(i + u - 1, j + v - 1) k(u, v).$$

Une couche de convolution comportant F filtres produit un tenseur de sortie

$$x^{(l)} \in \mathbb{R}^{F \times 28 \times 28}.$$

En effet, chaque filtre produit une carte de caractéristiques de la taille de l'image initiale. Pour la couche de convolution suivante, les K filtres auront pour dimension $F \times 3 \times 3$ afin de traiter toutes les cartes de caractéristiques, et ils produiront K cartes de caractéristiques.

Concernant les poids des éléments du filtre, ceux-ci ne sont pas décidés arbitrairement mais ils sont appris lors de la phase d'apprentissage du réseau de neurones.

Fonction de normalisation Chaque couche de convolution est suivie d'une fonction de normalisation afin de stabiliser l'entraînement du réseau, éviter d'obtenir des gradients qui explosent ou qui disparaissent, et ainsi réduire la sensibilité aux variations d'échelle dans les activations.

Fonction d'activation Cette fonction est appliquée après chaque couche de convolution afin d'introduire de la non-linéarité. Cela permet au modèle de détecter des relations non-linéaires et des motifs plus complexes dans les images.

Couches de mise en commun Le but des couches de mise en commun est d'extraire les informations principales de la matrice convoluée, afin d'éviter l'over-fitting et réduire la consommation de mémoire. On réduit donc la dimension de la matrice convoluée en appliquant une fonction d'aggrégation telle que le Max Pooling (garder la caractéristique maximale) ou la Mean Pooling (garder la moyenne des caractéristiques). La dernière couche de mise en commun est aplatie afin de pouvoir créer les couches entièrement connectées.

Couches entièrement connectées (FC) Ces couches sont les couches finales du réseau. Soit le vecteur aplati obtenu après les couches convolutionnelles et de pooling : $\mathbf{h} \in \mathbb{R}^D$. L'objectif des couches entièrement connectées (FC) est de produire un vecteur de logits : $\mathbf{z} \in \mathbb{R}^C$ (pour nous, $C = 47$ car EMNIST possède 47 classes). Afin de faire ça, chaque couche FC réalise une transformation linéaire de la forme :

$$\mathbf{z} = W\mathbf{h} + \mathbf{b},$$

où :

$W \in \mathbb{R}^{C \times D}$ est la matrice de poids,

$\mathbf{b} \in \mathbb{R}^C$ est le vecteur de biais.

Ensuite, le vecteur de probabilités attendu est obtenu grâce à une fonction softmax.

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

Infrastructure globale Maintenant que nous avons explicité les différents éléments d'un CNN, voici le schéma global d'un CNN :

$$x \xrightarrow{\text{Conv} + \text{ReLU}} x^{(1)} \xrightarrow{\text{Pooling}} x^{(2)} \xrightarrow{\text{Conv} + \text{ReLU}} \dots \xrightarrow{\text{Flatten}} \mathbf{h} \xrightarrow{\text{FC}} z \xrightarrow{\text{Softmax}} p.$$

3.3 Choix d'implémentations

Nous avons donc implémenté un CNN grâce au module `Pytorch` de python, en utilisant les fonctions déjà existantes de cette librairie. Afin de déterminer les différents paramètres ainsi que la structure de notre réseau neuronal, nous avons décidé de partir d'une structure simple. Ensuite, nous avons fait évoluer les paramètres afin d'améliorer la précision de notre classifieur sur le test set d'EMNIST `balanced`. Pour ce premier classifieur, nos choix ont donc été faits de manière empirique.

Paramètre	Choix d'implémentation et justification
Couches de convolution	6 couches <code>nn.Conv2d</code> avec <code>kernel_size=3</code> et <code>padding=1</code> pour conserver la taille. Nombre de canaux croissant de 1 à 128.
Fonction de normalisation	Application systématique de <code>nn.BatchNorm2d(num_features)</code> après chaque convolution. Stabilise l'entraînement, accélère la convergence et réduit la variance interne.
Fonction d'activation	Utilisation de <code>ReLU</code> : $f(x) = \max(0, x)$. Simple, peu coûteuse, crée de la sparsité et limite le problème de vanishing gradient.
Pooling	Utilisation de <code>nn.MaxPool2d(2,2)</code> pour réaliser un Max Pooling. Réduit la dimension tout en conservant l'information pertinente.
Couches entièrement connectées	Deux couches intermédiaires de taille 256 puis 128. Couche finale de taille 47 puis transformation en probabilités via SoftMax.

TABLE 1 – Résumé des choix d'implémentation du premier classifieur CNN

Formule de convolution Mathématiquement, pour une entrée avec C_{in} canaux, et une sortie avec C_{out} canaux, cela donne :

$$\text{out}(C_{out,j}) = \sum_{k=0}^{C_{in}-1} \text{weight}(C_{out,j}, k) \star \text{input}(k),$$

où $\star$ est l'opération de convolution définie plus haut. Chaque canal de sortie est obtenu par la somme des convolutions entre chaque canal d'entrée et son filtre associé.

Batch Normalization La fonction est `BatchNorm2d` : `nn.BatchNorm2d(num_features)`, qui effectue la normalisation par batch. L'argument `num_features` est égal aux nombre de canaux sortants de l'opération de convolution précédente. L'opération effectuée est la suivante :

$$y = \frac{x - E[x]}{\sqrt{\text{Var}[x] + \epsilon}} \cdot \gamma + \beta$$

Où :

- x est la valeur d'entrée.
- ϵ est une constante assez petite ajoutée à la variance de x afin d'éviter une division par zéro.
- γ est un paramètre appris par le réseau pendant l'entraînement permettant de modifier l'échelle des données.
- β est un paramètre utilisé pour décaler la moyenne des données

Pooling La fonction qui nous a permis de réaliser le Max Pooling est la suivante : `nn.MaxPool2d(2,2)`, en divisant chaque dimension de la matrice par 2 :

$$\text{MaxPool}(x)(i, j) = \max_{\substack{u \in \{0,1\} \\ v \in \{0,1\}}} x(2i + u, 2j + v).$$

3.4 Entraînement

L'entraînement d'un classifieur est un processus itératif qui permet de définir tous les paramètres des couches du réseau neuronal. En voici les étapes :

Premièrement, un dataset d'entraînement est importé. Quelques modifications peuvent être appliquées aux images si cela est jugé utile. Nous avons choisi d'appliquer une rotation de -90° ainsi qu'une symétrie orthogonale afin de remettre les images droites (les images du dataset d'origine ne le sont pas). Nous effectuons également du data augmentation. En particulier, une petite perturbation aléatoire (rotation, translation, cisaillement) est aussi appliquée afin d'améliorer la robustesse du classifieur.

Dans un second temps, ce dataset est parcouru un certain nombre de fois (le nombre d'époques). Les images ne sont pas "lues" une par une, mais en petit groupe (batch) de taille définie (le batch size). Nous avons utilisé un nombre d'époques de 30, et un batch size égal à 64.

Chaque batch "passe" donc dans le réseau neuronal et l'erreur commise (entre la prédiction et le vrai label) est calculée grâce à une fonction de perte. Ensuite, le gradient de cette fonction de perte est calculé par rapport à tous les paramètres du réseau neuronal, par backpropagation. Ce gradient représente, pour chaque paramètre, dans quelle direction il faudrait le faire varier pour réduire l'erreur. Les paramètres sont finalement actualisés par un optimizer et son scheduler.

3.4.1 Fonction de perte

La fonction de perte que nous avons utilisée est la Cross Entropy Loss améliorée avec une option de lissage d'étiquettes, c'est-à-dire que nous remplaçons les étiquettes one-hot parfaites (1 pour la bonne classe, 0 pour les autres) par des étiquettes légèrement adoucies : `nn.CrossEntropyLoss(label_smoothing=0.1)`. La perte pour un échantillon n est :

$$l_n = - \sum_{c=1}^C w_c \log \left(\frac{\exp(x_{n,c})}{\sum_{i=1}^C \exp(x_{n,i})} \right) y_{n,c}$$

Où :

- $x_{n,c}$ sont des logits (sortie de `forward_training`),
- $y_{n,c}$ sont les étiquettes, ce n'est donc plus 1 pour la bonne classe, mais 0.9 grâce au lissage. Pour toutes les autres classes ($c \neq \text{target}$), $y_{n,c}$ vaut environ 0.002 (les 0.1 restants sont divisés entre les 46 autres classes).

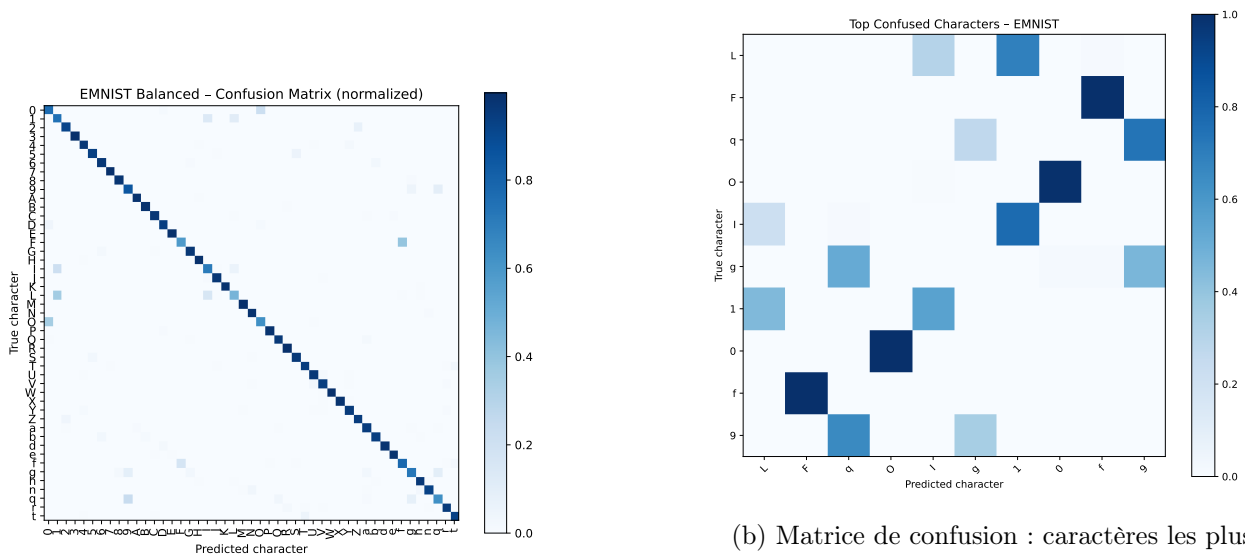
3.4.2 Optimiseur et Scheduler

L'optimiseur que nous avons retenu est Adam parce que celui-ci présentait les meilleurs résultats comparé à RMSProp et SGD par exemple. L'avantage de Adam est sa rapide convergence, son utilisation est très répandue dans les tâches de classification d'images. Notre choix s'est basé sur une

analyse empirique durant laquelle nous avons testé plusieurs possibilités et choisi celle qui donnait la plus haute précision. En paramètre de notre optimiseur, nous avons sélectionné un learning rate initial d'une valeur de $lr=0.001$, qui est la valeur recommandée par défaut. Le learning rate est le pas de gradient effectué. Celui-ci varie au cours du temps grâce au scheduler que nous avons utilisé : `optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.5)`. Il permet de réduire de moitié le learning rate tous les 5 epochs. L'idée est d'avoir des grands pas au début afin d'apprendre rapidement, et d'en faire des petits à la fin pour un meilleur raffinement.

3.5 Résultats et analyse

Pour cette première version de notre classifieur, nous obtenons une précision de 90,6% (calculée sur le test set avec un dataset non bruité). Nous avons réalisé la **matrice de confusion** sur les 47 classes, mais la trouvant peu lisible, nous avons également repris les caractères qui sont le plus souvent mal classifiés dans une plus petite matrice de confusion (voir Figure 1). À noter que nous avons volontairement décidé d'éliminer la diagonale de cette deuxième matrice afin de pouvoir mettre les couleurs à l'échelle en fonction des éléments **non-diagonaux** uniquement. Inclure la diagonale aurait écrasé les variations de couleur au sein des éléments non-diagonaux. Sans surprise, les erreurs ont lieu sur des symboles fort proches, comme le F majuscule et le f minuscule, ou le chiffre 0 avec la lettre O majuscule. Cela montre que notre réseau de neurones reconnaît correctement les formes, les courbes et les bords, c'est l'étape d'identification du caractère qui bloque.



(a) Matrice de confusion

(b) Matrice de confusion : caractères les plus souvent confondus (renormalisée après retrait de la diagonale afin de mieux visualiser)

FIGURE 1 – Matrices de confusion du premier classifieur

Concernant la phase d'**entraînement**, nous avons réalisé des graphes qui mettent en évidence l'apprentissage du modèle au fil des **epochs** (voir Figure 2). La fonction de perte diminue de manière globalement monotone, ce qui est cohérent avec l'objectif de minimisation : c'est cette métrique que l'algorithme cherche explicitement à minimiser. La précision augmente de manière globale, mais nous remarquons quelques fluctuations intermédiaires où elle diminue avant de remonter. Ce comportement est normal puisque la précision est une métrique plus instable, un petit changement dans les poids qui améliore la perte peut faire basculer la frontière de décision de certaines classes. De plus, l'algorithme ne cherche pas explicitement à augmenter cette métrique. Toutefois, les variations locales ne remettent pas en cause la tendance générale de l'apprentissage qui reste positive.

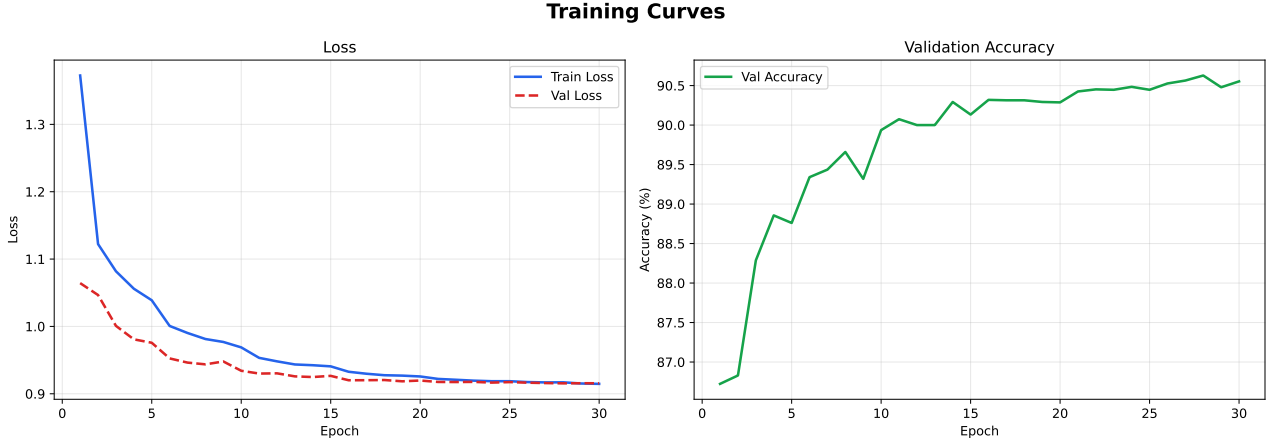


FIGURE 2 – Courbes lors de l'apprentissage du modèle

Nous avons également voulu analyser la robustesse de notre premier classifieur malgré le fait qu'il n'est pas encore optimisé dans ce sens. Pour ce faire, nous avons généré, pour chaque image $x \in [0, 1]^{28 \times 28}$, une perturbation δ de la manière suivante :

1. Sélection d'un sous-ensemble $S \subset \{1, \dots, 784\}$ de taille $|S| = k$, uniformément au hasard (sans remise).
2. Pour chaque $j \in S$, $v_j \sim \mathcal{U}(0, 1)$ et $\delta_j = v_j$. Pour $j \notin S$, $\delta_j = 0$.
3. Contrainte de validité des pixels (limite sur l'attaque autorisée) :

$$\delta_j \leftarrow \min(\delta_j, 1 - x)$$

de sorte que $x + \delta \leq 1$.

4. Projection sur la boule ℓ_2 de rayon ε :

$$\delta \leftarrow \begin{cases} \delta, & \|\delta\|_2 \leq \varepsilon, \\ \frac{\varepsilon}{\|\delta\|_2} \delta, & \|\delta\|_2 > \varepsilon. \end{cases}$$

L'image corrompue est alors

$$x_{\text{cor}} = \text{clip}(x + \delta, 0, 1).$$

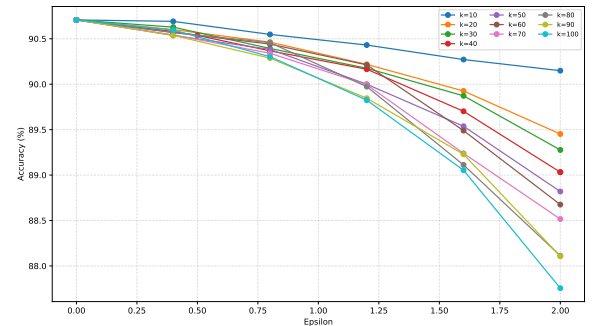


FIGURE 3 – Précision du classifieur pour un bruit ϵ , réparti sur un nombre de pixel k

Interprétation. La perturbation est non négative et contrainte par $\|\delta\|_2 \leq \varepsilon$. À cause du clipping et de la projection ℓ_2 , la distribution finale du bruit n'est ni gaussienne, ni uniformément distribuée en amplitude. Concernant les résultats de la Figure 3, nous remarquons que notre classifieur réagit de manière attendue au bruit, c'est-à-dire que ses performances baissent plus la norme du bruit est élevée. Nous remarquons également que son fonctionnement est davantage diminué lorsque le bruit est réparti sur un grand nombre de pixels. Cette deuxième découverte est moins intuitive et permet de nous donner un indice sur le type d'attaquant qui pourrait être plus efficace pour tromper les classifieurs.

3.6 Autres pistes envisagées

Nous avons également envisagé un classifieur MLP, que nous avons donc implémenté. Celui-ci était fonctionnel mais il présentait dès le début de moins bons résultats que le classifieur CNN, nous ne l'avons donc pas retenu. Nous avons cherché dans la littérature la raison pour laquelle un CNN est plus adapté dans notre contexte. Grâce à l'article [4] de nos références, nous avons compris que la différence principale entre un classifieur MLP et un classifieur CNN est le fait que le MLP est entièrement connecté entre chaque couche tandis que le CNN utilise la convolution ce qui permet de conserver une structure 2D au fil des couches. Le MLP est donc plus adapté à des données aux structures simples comme des tableaux 1D, ce qui explique le fait que CNN performe mieux dans notre cas.

4 Attaquants

4.1 Description du modèle d'attaque : perturbation ℓ_2

Un attaquant cherche à tromper un classifieur en ajoutant une perturbation imperceptible δ à une image x . On travaille avec le modèle de perturbation ℓ_2 :

Définition 1 (Boule de perturbation ℓ_2). Soit un budget $0 < \varepsilon \leq 2$, l'ensemble des perturbations admises est

$$\mathcal{B}_2(x, \varepsilon) = \{\tilde{x} \in [0, 1]^d \mid \|\tilde{x} - x\|_2 \leq \varepsilon\}.$$

Définition 2 (Norme ℓ_2). La norme ℓ_2 est définie comme :

$$\|v\|_2 = \sqrt{\sum_{i=1}^d v_i^2}$$

L'adversaire résout le problème de *maximisation interne* :

$$\delta^* = \arg \max_{\delta: \|\delta\|_2 \leq \varepsilon, x+\delta \in [0,1]^d} \mathcal{L}(f_\theta(x + \delta), y), \quad (2)$$

où $\mathcal{L}$ est la perte entropie croisée :

$$\mathcal{L}(f_\theta(x), y) = -\log p_y(x; \theta).$$

Notre attaquant est une attaque itérative de type PGD- ℓ_2 (*Projected Gradient Descent*), adaptée aux contraintes du projet. À partir d'une image $x \in [0, 1]^{28 \times 28}$, d'un classifieur TorchScript et d'un budget $\varepsilon \in [0, 2]$, il cherche une perturbation δ qui modifie la prédiction du classifieur tout en respectant

$$\|\delta\|_2 \leq \varepsilon, \quad x + \delta \in [0, 1]^{28 \times 28}.$$

(composante par composante). Ainsi, l'ensemble admissible est

$$\mathcal{C}(x, \varepsilon) = \left\{ \delta \in \mathbb{R}^d \mid -x \leq \delta \leq 1 - x, \|\delta\|_2 \leq \varepsilon \right\}, \quad (3)$$

où $d = 28 \times 28$.

4.2 Objectif de l'attaque

L'attaquant n'a pas accès au vrai label de l'image. Il part donc de la prédiction du classifieur sur l'image propre :

$$y_{\text{ref}} = \arg \max_k p_k(x),$$

où $p_k(x)$ désigne la probabilité prédite pour la classe k .

Attaque non ciblée. On cherche d'abord à faire baisser la confiance dans la classe prédite initialement. Pour cela, on maximise

$$J_{\text{unt}}(x + \delta) = \log\left(\max_{k \neq y_{\text{ref}}} p_k(x + \delta)\right) - \log(p_{y_{\text{ref}}}(x + \delta)). \quad (4)$$

Si cette quantité devient positive, une autre classe devient plus probable que la classe de référence.

Attaque ciblée. Dans une seconde étape, on peut aussi pousser l'image vers une classe cible $y_{\text{target}} \neq y_{\text{ref}}$ en maximisant

$$J_{\text{tar}}(x + \delta) = \log(p_{y_{\text{target}}}(x + \delta)) - \log(p_{y_{\text{ref}}}(x + \delta)). \quad (5)$$

4.3 Principe de l'algorithme

L'attaque suit une montée de gradient projetée. À l'itération t , on calcule le gradient de l'objectif choisi :

$$g^{(t)} = \nabla_{\delta^{(t)}} J(x + \delta^{(t)}).$$

Comme la perturbation peut être positive ou négative, on conserve directement le gradient complet. Celui-ci est ensuite normalisé en norme ℓ_2 :

$$\hat{g}^{(t)} = \frac{g^{(t)}}{\max(\|g^{(t)}\|_2, \eta)}, \quad \eta = 10^{-12}.$$

Pour stabiliser les mises à jour, on utilise un coefficient de **momentum** :

$$v^{(t+1)} = 0.85 v^{(t)} + \hat{g}^{(t)},$$

puis $v^{(t+1)}$ est lui aussi normalisé.

La mise à jour prend alors la forme

$$\tilde{\delta}^{(t+1)} = \delta^{(t)} + \alpha_t \hat{v}^{(t+1)}, \quad (6)$$

où α_t est un pas décroissant linéairement au cours de la phase :

$$\alpha_t = \alpha_0 \left(1 - \frac{1}{2} \frac{t}{T-1}\right).$$

Après chaque mise à jour, on reprojette la perturbation sur l'ensemble admissible (3) :

1. on impose $\delta \geq -x$;
2. on impose $\delta \leq 1 - x$;
3. si $\|\delta\|_2 > \varepsilon$, on renormalise :

$$\delta \leftarrow \delta \cdot \frac{\varepsilon}{\|\delta\|_2}.$$

Cette projection garantit que la perturbation retournée respecte toujours les contraintes.

4.4 Stratégie utilisée

L'attaque complète se déroule en trois phases :

1. **phase non ciblée depuis $\delta = 0$;**
2. **phase non ciblée avec départ aléatoire admissible ;**
3. **phase ciblée** vers quelques classes probables autres que y_{ref} .

À chaque phase, l'algorithme s'arrête dès que la prédiction change, c'est-à-dire lorsque

$$\arg \max_k p_k(x + \delta) \neq y_{\text{ref}}.$$

S'il n'obtient pas de succès complet, il conserve malgré tout la meilleure perturbation rencontrée au sens de l'objectif optimisé.

4.5 Hyperparamètres utilisés

Hyperparamètre	Valeur	Rôle / justification
Temps maximal par image	8.8 s	Marge de sécurité sous la limite de 10 s.
Coefficient de momentum β	0.85	Stabilise les directions de mise à jour et évite des oscillations trop fortes.
Seuil de stabilité numérique η	10^{-12}	Évite les divisions par zéro lors de la normalisation des gradients et des vitesses.
Nombre de classes cibles testées	3	On ne teste que les trois classes alternatives les plus probables pour limiter le coût temporel.
Phase 1 : attaque non ciblée depuis $\delta = 0$		
Nombre d'itérations T_1	14	Phase principale d'exploration, la plus longue.
Facteur de pas s_1	2.6	Donne un pas initial assez agressif pour quitter rapidement la région de l'image propre.
Phase 2 : attaque non ciblée avec redémarrage aléatoire		
Nombre d'itérations T_2	12	Légèrement plus court que la phase 1, car il sert surtout à échapper à un optimum local.
Facteur de pas s_2	2.2	Pas un peu plus conservateur que dans la phase 1.
Phase 3 : attaque ciblée		
Nombre d'itérations par cible T_3	8	Phase plus courte afin de respecter la contrainte de temps globale.
Facteur de pas s_3	2.0	Pas plus modéré, car la recherche ciblée est plus fine.
Nombre de cibles testées	3	On attaque les trois classes les plus probables autres que la classe de référence.

TABLE 2 – Hyperparamètres retenus pour l'attaquant PGD- ℓ_2

4.6 Résultats et analyse de l'attaquant PGD

Taux de succès des attaques La Figure 4 nous montre la robustesse des différents classifieurs rendus en S4 face à notre attaquant PGD. Nous remarquons que certains sont plus robustes que d'autres même si, pour un epsilon proche de 2, tous les classifieurs sont à moins de 20% de précision, ce qui est très mauvais. Néanmoins, ces résultats sont positifs vis-à-vis de notre attaquant qui est donc capable de tromper tous les classifieurs sans exception.

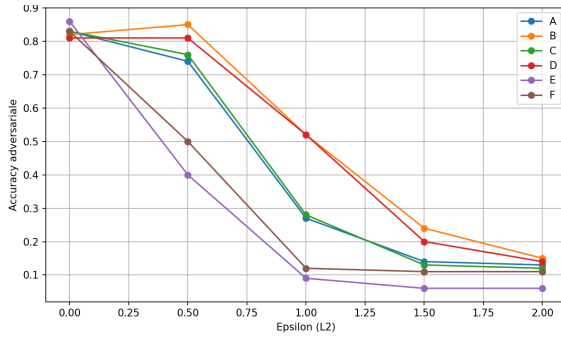


FIGURE 4 – Accuracy des classifieurs selon epsilon contre attaque PDG

Métrique	Valeur
Min	4.712 ms
Max	208.316 ms
Moyenne	141.129 ms
Écart-type	41.579 ms
Débit	7.1 att./s

TABLE 3 – Benchmark du temps d'attaque – `attacker_S11` sur EMNIST Balanced ($\epsilon_{L2} = 1.0$, 50 mesures)

Temps de run Les métriques principales sur notre temps de génération d'une perturbation sont reprises dans le Tableau 3. Nous sommes donc bien en deca du temps de run maximal autorisé de 10 secondes, ce qui montre que notre attaquant est plutôt rapide, il produit plus de 7 attaques par seconde.

4.7 Second attaquant envisagé : Deepfool

4.7.1 Principe

DeepFool adopte une approche différente de FGSM ou PGD. Au lieu de suivre le gradient d'une fonction de perte, cette attaque cherche directement à trouver la plus petite perturbation possible qui permet de franchir la frontière de décision du classifieur. C'est une attaque itérative qui à chaque itération, linéarise le classifieur autour de l'image, calcule la distance minimale à une autre classe et ajoute la perturbation correspondante à l'image. Ce processus est répété jusqu'à ce que la classe prédite change, que le nombre d'itérations maximal soit atteint ou que la contrainte de temps soit dépassée. Cette méthode est souvent meilleure que FGSM pour des petits epsilon bien qu'elle prenne plus de temps et a l'avantage de donner des attaques invisibles à l'œil nu comme on recherche une perturbation minimale.

4.7.2 Explication de notre implémentation

À partir de l'image initiale x , on calcule la classe prédite y avec le classifieur que nous désirons attaquer. Nous considérerons ici que le classifieur prédit parfaitement la classe d'une image non-bruitée. La perturbation est initialisée à zéro, puis mise à jour progressivement.

À chaque itération, une nouvelle image perturbée est construite :

$$x_{adv} = x + \delta$$

Le modèle est alors linéarisé localement autour de x_{adv} en utilisant les gradients des scores de sortie. Pour chaque classe $k \neq y$, on calcule :

$$w_k = \nabla f_k(x_{adv}) - \nabla f_y(x_{adv})$$

et

$$f_k = f_k(x_{adv}) - f_y(x_{adv})$$

Ces quantités permettent d'estimer la distance à la frontière de décision séparant la classe actuelle y d'une autre classe k . Cette distance est approximée par :

$$\text{pert}_k = \frac{|f_k|}{\|w_k\|_2}$$

On sélectionne ensuite la classe k^* correspondant à la plus petite distance :

$$k^* = \arg \min_k \text{pert}_k$$

La perturbation est alors mise à jour dans la direction optimale :

$$\delta \leftarrow \delta + \frac{\text{pert}_{k^*}}{\|w_{k^*}\|_2} w_{k^*}$$

Après chaque mise à jour, une projection sur la boule L_2 est appliquée afin de garantir la contrainte :

$$\|\delta\|_2 \leq \epsilon$$

L'algorithme s'arrête lorsque la classe prédite change, lorsque le nombre maximal d'itérations est atteint, lorsque la contrainte de temps est dépassée ou lorsque $\|\delta\|_2 > \epsilon$. Dans ce dernier cas,

$$\delta \leftarrow \delta \cdot \frac{\epsilon}{\|\delta\|_2}.$$

Enfin, l'image adversariale est projetée dans l'intervalle valide $[-1, 1]$, et la perturbation finale est recalculée :

$$\delta = \text{clip}(x + \delta, -1, 1) - x$$

4.7.3 Hyperparamètres utilisés

Notre attaquant réalise maximum $N=20$ itérations par attaque pour tenter de changer la classe prédite. Cela nous permet d'avoir des attaques efficaces en un temps acceptable. Nous arrêtons une attaque lorsqu'elle prend plus de 9.5 secondes à se réaliser et renvoyons la perturbation δ actuelle afin de satisfaire la limite de 10s par attaque imposée par le projet.

4.7.4 Résultats et analyse

Dans la Figure 5, le graphique représente la précision des différents classifieurs selon le ϵ autorisé pour chaque attaque. Pour réaliser ce graphique, nous avons choisi 100 images aléatoirement dans le dataset EMNIST afin d'avoir un temps d'exécution acceptable. Les résultats sont satisfaisants : tous les classifieurs sont trompés par les attaques de Deepfool, et cela à une vitesse très similaire pour chacun d'eux. Aucun ne se distingue par une robustesse plus élevée.

Dans le Tableau 4, nous avons repris les métriques temporelles principales pour l'attaquant Deepfool. Nous remarquons que cet attaquant est également plutôt rapide, en tous cas il respecte largement le maximum autorisé de 10 secondes. Son taux d'attaque est légèrement plus faible que PGD, il est en effet un peu plus lent.

Métrique	Valeur
Min	120.323 ms
Max	1679.106 ms
Moyenne	180.952 ms
Médiane	149.873 ms
Écart-type	141.453 ms
Débit	5.5 attaques/s

TABLE 4 – Benchmark du temps d'attaque – Deepfool sur EMNIST Balanced

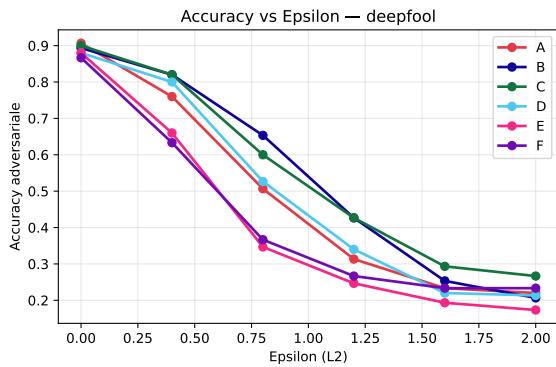


FIGURE 5 – Précision des modèles remis en S4 contre l’attaquant Deepfool

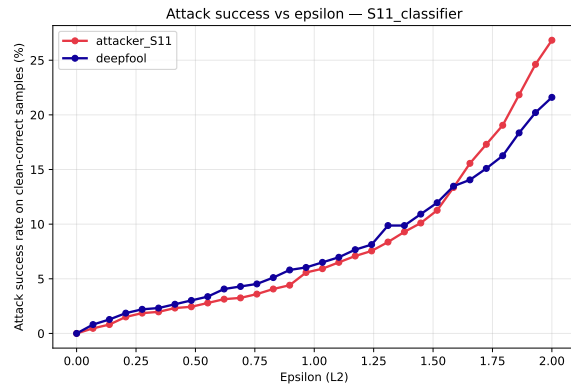


FIGURE 6 – Comparaison du taux de succès selon epsilon de Deepfool et PGD

4.8 Comparaison PGD vs Deepfool

Comme nous nous y attendions, Deepfool et PGD donnent des résultats relativement identiques pour de petits epsilons mais le taux de succès de Deepfool devient assez limité pour de grands ϵ contrairement à celui de PGD qui augmente beaucoup pour $\epsilon \geq 1.6$, dépassant Deepfool. Cela ajouté au fait que PGD est plus rapide que Deepfool, nous a convaincu de le choisir comme attaquant pour notre projet.

5 Deuxième classifieur

5.1 Description des améliorations techniques

Pour le deuxième classifieur, nous avons amélioré plusieurs aspects. Dans un premier lieu, nous avons effectué du training avec des images qui ont déjà été attaquées, ce qui augmente la robustesse du modèle aux attaques. Certains paramètres du modèle ont aussi été revus tels que la normalisation utilisée ainsi que l’optimiseur.

5.1.1 Adversarial training

Pour créer un classifieur robuste aux attaques adversariales, on utilise l’entraînement adversarial par PGD introduit dans la référence [1]. Contrairement à ce qu’une approche naïve pourrait suggérer, il ne s’agit pas de deux phases séquentielles (d’abord entraîner le modèle, puis l’attaquer), mais d’un processus entrelacé à chaque batch d’entraînement.

Concrètement, à chaque étape, on commence par geler les poids θ du modèle et on lance T itérations de PGD pour construire, pour chaque image x du batch, la perturbation δ^* qui maximise la perte — c’est-à-dire l’exemple adversarial le plus nuisible possible pour le modèle dans son état actuel. On obtient ainsi une version attaquée $x_{\text{adv}} = x + \delta^*$, contrainte à rester dans la boule $\|\delta\|_2 \leq \epsilon$ et dans la plage de pixels valides $[0, 1]$. Dans un second temps, on dégèle les poids et on effectue un pas de descente de gradient classique sur la perte calculée sur ces exemples adversariaux, afin que le modèle apprenne à les classer correctement.

L’attaquant s’adapte en permanence au modèle courant, et le modèle s’adapte en permanence aux attaques courantes. Au fil des epochs, les perturbations générées deviennent de plus en plus sophistiquées car le modèle se renforce, ce qui force ce dernier à développer des représentations internes plus robustes plutôt que de mémoriser des exemples adversariaux fixes. La méthode PGD précise utilisée pour générer ces perturbations est détaillée dans l’annexe C.1 (celle-ci est légèrement différente de celle utilisée pour notre attaquant, même si le principe derrière reste très similaire).

Objectif d'entraînement adversarial La minimisation du risque empirique standard minimise :

$$\min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}} [\mathcal{L}(f_{\theta}(x), y)].$$

L'entraînement adversarial résout plutôt le problème minimax :

$$\min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}} \left[\max_{\|\delta\|_2 \leq \varepsilon} \mathcal{L}(f_{\theta}(x + \delta), y) \right] \quad (7)$$

Le problème max à l'intérieur correspond à l'attaque PGD utilisée : elle essaie de maximiser la fonction de perte du modèle (i.e. elle cherche la meilleure perturbation possible). Le min à l'extérieur, lui, cherche les paramètres qui, en moyenne, obtienne la moyenne de perte la plus faible possible sur les attaques sur le domaine $\mathcal{D}$.

Dans le code, une combinaison convexe de la perte propre et de la perte adversariale est utilisée :

$$\mathcal{L}_{\text{total}} = (1 - \lambda) \mathcal{L}(f_{\theta}(x), y) + \lambda \mathcal{L}(f_{\theta}(x + \delta^*), y) \quad (8)$$

Cela permet de bénéficier d'une grande robustesse du modèle entraîné sur le PGD, mais aussi de conserver la bonne précision sur les images non-attaquées développée pour le premier classifieur rendu.

Lissage des étiquettes. La perte entropie croisée utilise des cibles adoucies avec un paramètre de lissage $\eta = 0,05$:

$$\mathcal{L}_{\eta}(f_{\theta}(x), y) = - \sum_{c=0}^{C-1} \tilde{y}_c \log p_c(x; \theta), \quad \tilde{y}_c = \begin{cases} 1 - \eta + \frac{\eta}{C} & \text{si } c = y, \\ \frac{\eta}{C} & \text{sinon.} \end{cases} \quad (9)$$

Cela permet de laisser une marge sur la probabilité que le modèle renvoie pour une certaine classe. On ne pénalise ni une probabilité légèrement éloignée de 1 ni les probabilités des autres classes légèrement différentes de 0.

Calendrier d'entraînement

Phase 1 : ($e < 2$). Durant les deux premiers époques, aucune perturbation adversariale n'est générée ($\varepsilon_{\text{train}} = 0$). Le modèle est entraîné sur des données propres afin d'établir une initialisation stable.

Phase 2 : ($2 \leq e < 6$). Le rayon d'entraînement est augmenté linéairement sur quatre époques :

$$\varepsilon_e = \varepsilon_{\text{max}} \cdot \min\left(1, \frac{e - 2 + 1}{4}\right), \quad \varepsilon_{\text{max}} = 2.0. \quad (10)$$

Décroissance du taux d'apprentissage. Un scheduler divise le taux d'apprentissage (learning rate) par deux tous les 6 époques :

$$\eta_e = \eta_0 \cdot (0.5)^{\lfloor e/6 \rfloor}, \quad \eta_0 = 10^{-3}.$$

Le grand pas au début permet de se rapprocher rapidement de la solution, mais pour éviter d'osciller autour après plusieurs époques, on diminue la taille du pas.

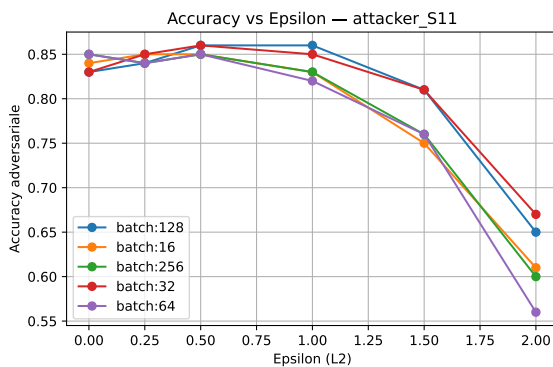
5.1.2 Evolution de l'architecture

Pour faire des différentes mesures dans cette section, nous avons fait l'hypothèse que les paramètres n'influent en général pas sur les autres, c'est-à-dire que chaque paramètre admet une valeur optimale qui est indépendante des autres valeurs de paramètres. Ainsi, nous considérons que si nous testons différentes valeurs pour un paramètre tout en gardant constants les autres paramètres, la valeur optimale pour ce paramètre restera optimale en changeant les autres. Afin d'améliorer notre premier classifieur, nous avons modifié son architecture de la sorte :

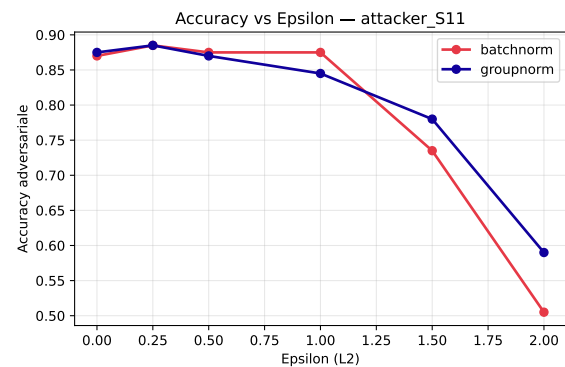
Couches de convolution et filtres Nous avons réduit notre nombre de couches à 4 couches (contre 6 initialement), mais augmenté progressivement le nombre de filtres lors de chaque convolution de manière plus marquée (32, 64, 96 et 128).

Batch size Nous avons testé plusieurs tailles de batch pour évaluer leur impact sur la précision. Une graphe qui reprend la précision des modèles faces à des attaques adversariales est repris à la Figure 7a. On observe que les tailles de batch qui produisent les meilleurs résultats sont les tailles de 32 et 128.

Normalisation Notre premier classifieur utilisait la Batch Normalization **BatchNorm2d**, qui normalisait les données par batch. Pour ce nouveau classifieur, nous avons opté pour la **GroupNorm** qui normalise les canaux par groupes au sein de chaque image, indépendamment des autres échantillons du lot.



(a) Impact de la batch size sur la précision des modèles



(b) Comparaison de la robustesse selon la norme utilisée

FIGURE 7 – Optimisation du deuxième classifieur

La figure 7b montre la précision en fonction du ϵ utilisé pour deux classifieurs avec une architecture identique, excepté la norme utilisée.

Nous observons que pour des données non perturbées ($\epsilon = 0$), les deux modèles ont une précision quasi identique. Le changement de normalisation n'affecte donc pas la performance de base du classifieur sur des images non attaquées.

Face aux faibles perturbations, les deux modèles restent performants et assez concurrents. Dès que l'attaque devient sévère, le modèle avec **BatchNorm** s'effondre beaucoup plus rapidement jusqu'à atteindre une précision de 50% à $\epsilon = 2$ contre 60% de précision pour la **GroupNorm**.

Cette supériorité s'explique par l'indépendance des calculs. Avec la **BatchNorm**, les images d'un même lot sont liées entre elles lors de la normalisation. Une seule image fortement attaquée peut donc fausser les calculs et nuire aux résultats d'autres images. Contrairement à la **GroupNorm** qui traite chaque image séparément. Comme les images ne sont plus liées, les attaques ne se propagent pas, ce qui stabilise le modèle même face à des perturbations plus fortes.

Valeur de λ (coefficient de momentum) Nos expériences (Figure 8) montrent qu'un coefficient de $\lambda = 0.6$ donne le meilleur compromis entre précision sans attaque et précision sous attaque adversariale.

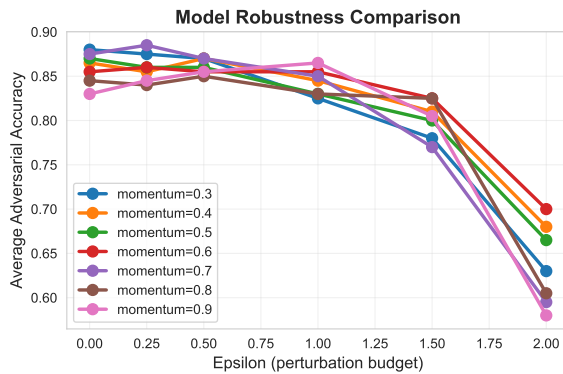


FIGURE 8 – Comparaison des différents λ

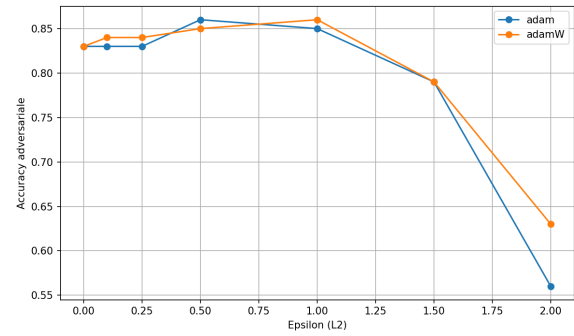


FIGURE 9 – Comparaison de deux modèles équivalents, l'un utilisant AdamW et l'autre Adam

Optimiseur Afin d'être plus efficace face aux attaques adversariales, nous avons opté pour l'optimiseur AdamW. Leur différence majeure réside dans la manière dont ils gèrent la décroissance poids (technique permettant d'empêcher les paramètres du modèle de devenir trop grands) : dans l'optimiseur Adam, cette décroissance est combinée avec le calcul du gradient, contrairement à AdamW qui effectue l'un et puis seulement l'autre. La mise à jour des poids est donc beaucoup plus stable. Ce qui permet une meilleure généralisation des données afin de minimiser la sensibilité au bruit.

Comme nous pouvons l'observer dans la Figure 9, les deux modèles ont une performance assez similaire pour des valeurs de $\epsilon \leq 1.5$, et comme attendu, l'optimiseur AdamW présente une meilleure stabilité pour ces même valeurs de ϵ . C'est pour des perturbations plus conséquentes ($\epsilon \geq 1.5$) que l'optimiseur AdamW va se montrer beaucoup plus efficace avec une précision d'environ 63% pour $\epsilon = 2$, contre 56% pour l'optimiseur Adam.

Epsilon de training L'analyse empirique de la Figure 10 nous permet de constater que l' $\epsilon_{\text{training}}$ idéal est de 1,5 ou 2,0.

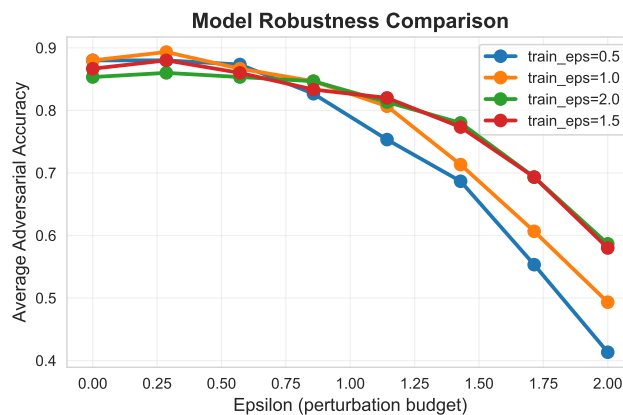


FIGURE 10 – Précision selon ϵ pour différents $\epsilon_{\text{training}}$

5.2 Hyperparamètres finaux du classifieur

Hyperparamètre	Valeur
Général	
Taille de batch	32
Nombre d'épochs	20
Optimiseur (AdamW)	
Taux d'apprentissage initial	10^{-3}
Poids de régularisation λ	5×10^{-4}
Réduction du taux (StepLR)	$\times 0.5$ tous les 6
Fonction de perte	
Lissage des étiquettes	0.05
Poids de la perte adversariale α	0.6
Entraînement adversarial (PGD-ℓ_2)	
Budget de perturbation $\varepsilon_{\text{train}}$	1.5
Nombre d'itérations PGD	6
Nombre de redémarrages	1
Montée en puissance de ε	linéaire sur 4 epoch

TABLE 5 – Hyperparamètres retenus pour l'entraînement du classifieur

5.3 Résultats et analyse

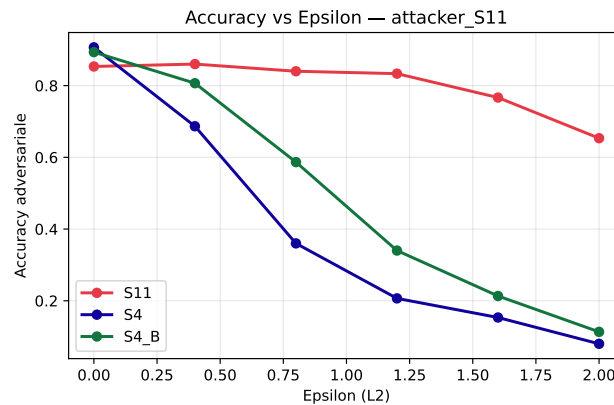
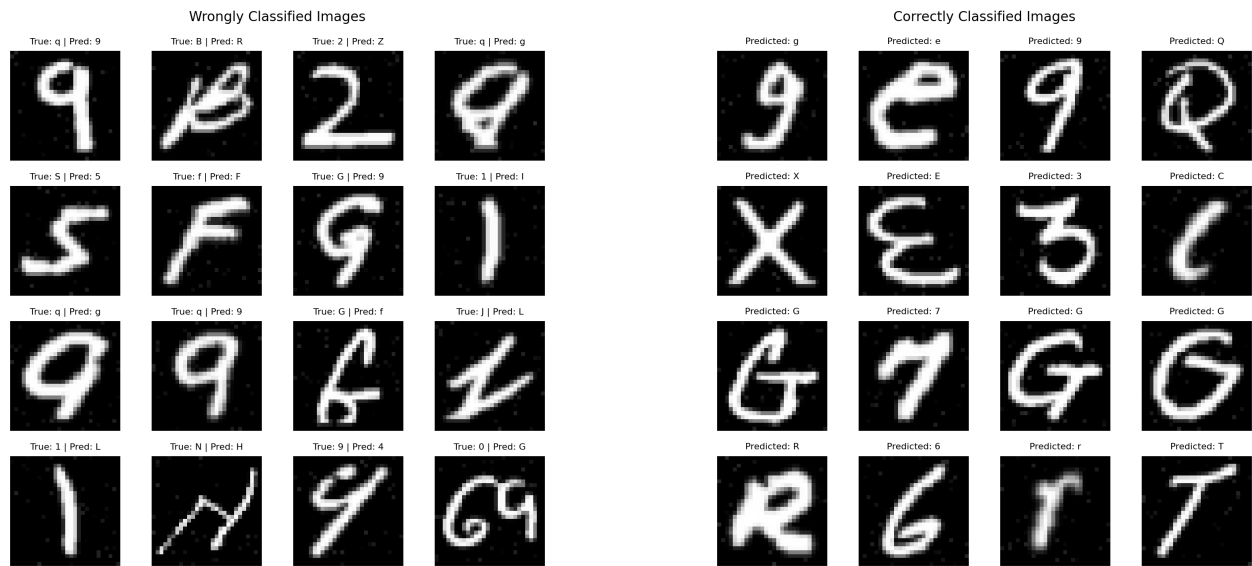


FIGURE 11 – Evolution de la précision des classifieurs en fonction de la taille de perturbation ϵ . Comparaison avec notre modèle de S4 et S11 et le meilleur modèle des autres groupes en S4 également.

Nous constatons que les classifieurs des groupes A et B (courbes bleue et orange) vont très vite se montrer vulnérables aux perturbations engendrées par ϵ . En effet, leur précision chute brusquement avant d'atteindre une valeur quasi nulle à $\epsilon = 2.0$. Ils sont donc assez sensibles aux petites variations de pixels que le modèle n'a jamais rencontré lors de son entraînement.

En ce qui concerne notre modèle final (courbe verte), celui-ci maintient une précision supérieure à 80% jusqu'à $\epsilon = 1.0$ et ne descendra pas en dessous de 50% même pour une attaque forte à $\epsilon = 2.0$.

Cette stabilité est due notamment au minimax (Eq. 7) : le fait de s'entraînant sur les pires cas possibles permet au modèle d'ignorer le bruit pour se focaliser sur des caractéristiques plus stables et fiables.



(a) Ensemble d'images incorrectement classifiées

(b) Ensemble d'images correctement classifiées

FIGURE 12

Images incorrectement classifiées : Nous constatons que les erreurs de classification sont souvent faites sur des caractères relativement ambigus (ex : un '9' qui ressemble à un 'g', ou un '2' à un z). Remarquons que certaines de ces confusions pourraient même être faites par des humains (voir Figure 15).

Analyse des résultats Top- k : La figure 13 présente la précision Top- k (pour $k \in \{1, 2, 3\}$) du modèle, définie comme le pourcentage de cas où la classe réelle figure parmi les k premières prédictions.

Observations. On observe une progression croissante de la précision avec k :

- **Top-1** (bleu) : **88,3%** — la classe réelle est la première prédiction dans près de neuf cas sur dix.
- **Top-2** (orange) : **96,9%** — la classe réelle figure parmi les deux premières prédictions dans la grande majorité des cas.
- **Top-3** (vert) : **99,1%** — la classe réelle est presque systématiquement présente dans les trois premières prédictions.

Interprétation. Le gain de précision entre Top-1 et Top-3 est d'environ **+11 %**, ce qui indique que le modèle, bien qu'hésitant parfois à placer la bonne classe en première position, l'identifie néanmoins parmi ses hypothèses les plus probables. Cet écart suggère une confusion résiduelle entre classes proches, que le modèle ne parvient pas toujours à lever avec certitude.

Le modèle atteint des performances globalement très satisfaisantes, avec un score Top-3 supérieur à 99%. Le taux Top-1 de 88,3% constitue la principale marge de progression, et une analyse des erreurs de classification pourrait permettre d'identifier les classes les plus ambiguës afin d'orienter les améliorations futures.

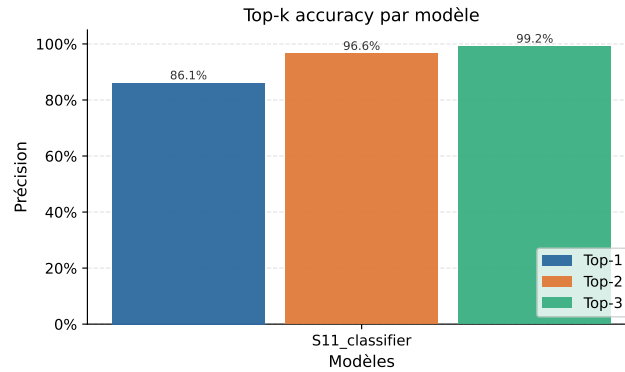


FIGURE 13 – Pourcentage de présence de la classe réelle dans les K premières classes

5.4 Étude comparative des classifieurs de S4 et de S11

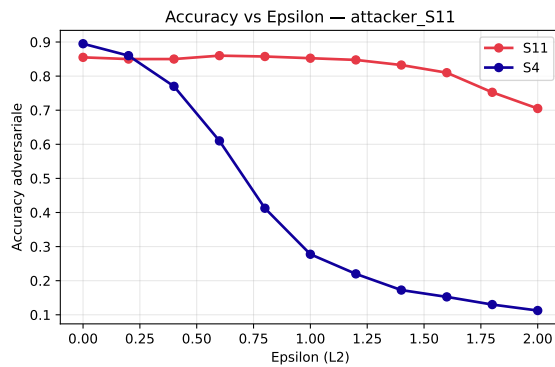


FIGURE 14 – Comparaison de la précision des classifieurs de S4 et de S11 face à un attaquant de type PGD

Modèle	Moyenne	Médiane	FPS ^a
S11	0.553 ms	0.524 ms	1809.0
S4	0.936 ms	0.890 ms	1068.6

TABLE 6 – Résumé comparatif des performances des modèles

^a. Frame Per Second : nombre d'images classifiées par seconde sur un ordinateur classique (sans GPU)

Analyse de la robustesse Bien que le classifieur S4 soit plus performant sur les images non-perturbées, sa précision va diminuer considérablement à mesure que la perturbation ϵ augmente. Cela s'explique par le fait que le réseau apprend des frontières de décisions nettes mais assez fragiles, et peut donc très vite se montrer vulnérable face à des petites variations. Concernant notre second modèle, celui-ci met en avant une très bonne stabilité, en maintenant une précision de 80% jusqu'à $\epsilon = 1.2$ environ. Notons que ce modèle montre une légère amélioration de la précision pour $0.4 \leq \epsilon \leq 0.6$. Ce qui met en avant l'efficacité de l'adversarial training : le modèle apprend à ignorer le bruit pour se focaliser exclusivement sur les traits importants.

Analyse des performances de calcul Cette table nous montre que notre second modèle (S11) est aussi plus efficace en terme de temps de traitement. Ce qui est tout à fait cohérent avec nos choix d'architecture (réduction du nombre de couches de convolution et augmentation du nombre de filtres), permettant ainsi d'optimiser le passage des données à travers le réseau. Il en résulte donc un débit d'images par seconde (FPS) nettement supérieur pour notre second classifieur (1809 contre seulement 1068.6 pour le premier).

Trade-off de la précision vis-à-vis de la robustesse Pour passer du classificateur de S_4 à celui de S_{11} , nous avons dû faire un compromis entre précision sur les images de tests. Celui-ci est une perte

de 3 – 4% sur les images des tests. Par contre nous gagnons 60% de précision sur les images attaquées par une méthode PGD.

Temps d'inférence Le temps d'inférence du modèle rendu est plus rapide sur le second classificateur.

6 Outils de calculs utilisés

Pour l'entraînement de nos modèles, nous avons utilisé nos ordinateurs personnels (sur `cpu` ou `cuda` si disponible). Nous avons également utilisé `Google Colab` qui permet d'avoir accès à de puissants GPU pour permettre d'entraîner nos modèles plus rapidement (notre expérience montre qu'on multiplie la vitesse d'entraînement par 2 ou 3).

7 Ajouts supplémentaires au projet

7.1 Programme de dessin interactif

Afin de mieux comprendre le comportement de nos classifieurs et de nos attaquants, nous avons développé un programme de dessin interactif permettant de tracer manuellement un caractère sur une grille de taille 28×28 , c'est-à-dire au même format que les images de la base EMNIST. L'utilisateur peut ainsi dessiner un chiffre ou une lettre directement à la souris, puis soumettre ce dessin aux différents modèles afin d'observer immédiatement leurs prédictions.

L'interface a été conçue comme un outil d'analyse qualitative. Elle permet d'abord de charger automatiquement plusieurs classifieurs sauvegardés au format TorchScript, puis de détecter pour chacun d'eux le format d'entrée attendu. En pratique, cela rend l'outil compatible avec plusieurs architectures sans devoir modifier le code à la main. Une fois le dessin réalisé, le programme affiche les prédictions de tous les modèles sélectionnés, ce qui permet de comparer facilement leur comportement sur une même entrée.

Nous avons également intégré un attaquant externe dans cette interface. Pour une valeur donnée de ε , choisie à l'aide d'un curseur, le programme génère une perturbation adversariale δ , construit l'image perturbée $x + \delta$, puis affiche, pour chaque modèle, la classe prédite avant et après attaque ainsi que la norme ℓ_2 de la perturbation. Un second curseur permet d'ajuster l'épaisseur du pinceau, ce qui rend le dessin plus souple et plus naturel. L'intérêt principal de cet outil est de fournir une visualisation immédiate des effets d'une attaque : il devient alors possible de voir non seulement si un modèle est trompé, mais aussi à quoi ressemble concrètement l'image adversariale produite.

Ce programme ne remplace évidemment pas les benchmarks systématiques réalisés sur la base de test, mais il constitue un complément utile. Il permet de développer une intuition sur les faiblesses des classifieurs, de vérifier rapidement le bon fonctionnement d'un attaquant, et d'illustrer de manière visuelle le phénomène d'attaque adversariale.

7.2 Recherche du plus petit ε adversarial

Nous avons ensuite développé une seconde version, plus avancée, du programme interactif. Son objectif n'est plus seulement de produire une attaque pour une valeur imposée de ε , mais de déterminer automatiquement le plus petit niveau de perturbation qui suffit à faire échouer un classifieur.

Le principe adopté est le suivant. À partir du dessin réalisé par l'utilisateur, le programme commence par calculer la prédiction du modèle en l'absence de perturbation. Il applique ensuite l'attaquant pour une suite de valeurs croissantes de ε , avec un pas grossier choisi par l'utilisateur. Dès qu'un changement de classe est observé, l'intervalle critique ainsi détecté est raffiné par dichotomie afin d'obtenir une estimation plus précise de la valeur minimale ε^* . Cette procédure permet d'évaluer de manière simple la robustesse relative des différents modèles sur une entrée donnée.

Cette version offre également un mode d'évaluation par rapport à un label imposé. L'utilisateur peut en effet renseigner le caractère qu'il pense avoir dessiné ; le programme cherche alors le plus petit ε pour lequel la prédiction n'est plus égale à ce label. Si aucun label n'est fourni, le critère utilisé est simplement le premier changement par rapport à la prédiction propre du modèle. Cette distinction est utile : dans certains cas, un modèle peut déjà se tromper sur l'image non perturbée, et il est alors plus pertinent de mesurer la robustesse par rapport au label attendu.

Enfin, le programme génère plusieurs visualisations complémentaires. Il affiche d'abord, pour chaque modèle, l'image adversariale minimale trouvée ainsi que la classe obtenue et la norme de la perturbation. Il produit ensuite un histogramme représentant les valeurs minimales ε^* , ce qui permet de comparer directement la robustesse des modèles. Une troisième figure montre l'évolution de l'état de la prédiction en fonction de ε , ce qui met en évidence le moment où survient le basculement vers l'erreur.

Cet outil fournit donc une analyse plus fine que la simple visualisation d'une attaque à ε fixé. Il permet de quantifier expérimentalement, sur des exemples dessinés à la main, la sensibilité de chaque classifieur aux perturbations adversariales, tout en conservant une interface simple et intuitive.

8 Conclusion

Ce projet de classification d'images a permis à notre équipe d'appréhender la complexité des réseaux de neurones convolutifs et de confronter leurs performances à des menaces adversariales. Au travers de ce parcours, nous avons relevé les défis majeurs qu'implique la classification robuste.

La première implémentation de notre classifieur a démontré l'efficacité des réseaux de neurones convolutifs pour la reconnaissance de caractères, atteignant une précision de 90% sur des images non perturbées. Cependant, c'est l'étape de conception de nos propres attaquants et ensuite de test de ceux-ci sur notre modèle qui a mis en avant sa fragilité face à des perturbations parfois imperceptibles à l'oeil nu. En effet, un apprentissage fondé uniquement sur la performance peut vite s'avérer être vulnérable face à des manipulations ciblées des pixels.

Enfin, pour aboutir à la version finale de notre classifieur, nous avons cherché à faire un compromis entre robustesse et précision. En intégrant l'entraînement adversarial et en optimisant l'architecture du modèle, nous avons réussi à le stabiliser face aux images bruitées les plus sévères. Ce travail prouve qu'il est bel et bien possible de renforcer la sécurité du modèle tout en améliorant son efficacité globale, offrant ainsi une solution fiable.

9 Utilisation de l'IA

Durant la réalisation de ce projet, des outils d'intelligence artificielle nous ont aidé pour réaliser certaines tâches listées ci-dessous.

- Brainstorming des différentes possibilités de classifieur et/ou d'attaquant
- Aide à la génération du squelette de notre code
- Aide à la compréhension de la littérature scientifique utilisée
- Aide au processus de débogage de notre code
- Aide à la réalisation de notre interface graphique

Les intelligences artificielles utilisées sont les suivantes : ChatGPT, Copilot, Gemini et Claude. Toutes les informations générées par celles-ci ont été testées, vérifiées ou confirmées par des sources extérieures.

Références

- [1] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu. *Towards Deep Learning Models Resistant to Adversarial Attacks*. ICLR, 2018.
- [2] Y. Wu et K. He. *Group Normalization*. ECCV, 2018.

- [3] N. Carlini et D. Wagner. *Towards Evaluating the Robustness of Neural Networks*. IEEE S&P, 2017.
- [4] Linda Joel. *Comparative Performance Analysis of CNN and MLP on CIFAR-10 Using Python : A Detailed Evaluation and Interpretation*. International Journal of Applied Mathematics, vol. 10, no. 1, 2025. Available at : <https://ijamjournal.org/ijam/publication/index.php/ijam/article/view/769>.
- [5] Practical DevSecOps. *Projected Gradient Descent (PGD) – Definition and Explanation*. Available at : <https://www.practical-devsecops.com/glossary/projected-gradient-descent-pgd/>.
- [6] PyTorch Team. *Adam Optimizer — PyTorch 2.11 Documentation*. Available at : <https://docs.pytorch.org/docs/stable/generated/torch.optim.Adam.html>.
- [7] Jianxin Wu. *Introduction to Convolutional Neural Networks*. Available at : <https://jasoncantarella.com/downloads/CNN.pdf>.

A Architecture du premier réseau de neurônes

Algorithme 1 FORWARD — Inférence du réseau

Entrées : Image $x \in [0, 1]^{28 \times 28}$ (ou batch $B \times 28 \times 28$)

Sortie : Vecteur de probabilités $y \in [0, 1]^{47}$ (ou $B \times 47$)

```

1: Prétraitement :
2: if  $x$  est de dimension (28, 28) then
3:    $x \leftarrow x.\text{unsqueeze}(0).\text{unsqueeze}(0)$  ▷ (1, 1, 28, 28)
4: else if  $x$  est de dimension ( $B$ , 28, 28) then
5:    $x \leftarrow x.\text{unsqueeze}(1)$  ▷ ( $B$ , 1, 28, 28)
6: end if
7: Bloc convolutif 1 :
8:  $x \leftarrow \text{ReLU}(\text{BN}_1(\text{Conv}_{1 \rightarrow 32}(x)))$  ▷ ( $B$ , 32, 28, 28)
9:  $x \leftarrow \text{ReLU}(\text{BN}_2(\text{Conv}_{32 \rightarrow 32}(x)))$  ▷ ( $B$ , 32, 28, 28)
10:  $x \leftarrow \text{ReLU}(\text{BN}_3(\text{Conv}_{32 \rightarrow 32}(x)))$  ▷ ( $B$ , 32, 28, 28)
11:  $x \leftarrow \text{MaxPool}_{2 \times 2}(x)$  ▷ ( $B$ , 32, 14, 14)
12: Bloc convolutif 2 :
13:  $x \leftarrow \text{ReLU}(\text{BN}_4(\text{Conv}_{32 \rightarrow 64}(x)))$  ▷ ( $B$ , 64, 14, 14)
14:  $x \leftarrow \text{ReLU}(\text{BN}_5(\text{Conv}_{64 \rightarrow 64}(x)))$  ▷ ( $B$ , 64, 14, 14)
15:  $x \leftarrow \text{MaxPool}_{2 \times 2}(x)$  ▷ ( $B$ , 64, 7, 7)
16: Bloc convolutif 3 :
17:  $x \leftarrow \text{ReLU}(\text{BN}_6(\text{Conv}_{64 \rightarrow 128}(x)))$  ▷ ( $B$ , 128, 7, 7)
18:  $x \leftarrow \text{MaxPool}_{2 \times 2}(x)$  ▷ ( $B$ , 128, 3, 3)
19: Classifieur :
20:  $x \leftarrow x.\text{reshape}(B, 1152)$ 
21:  $x \leftarrow \text{Dropout}_{0.25}(x)$ 
22:  $x \leftarrow \text{ReLU}(\text{FC}_1(x))$  ▷ 1152 → 256
23:  $x \leftarrow \text{Dropout}_{0.25}(x)$ 
24:  $x \leftarrow \text{ReLU}(\text{FC}_2(x))$  ▷ 256 → 128
25: logits  $\leftarrow \text{FC}_3(x)$  ▷ 128 → 47
26:  $y \leftarrow \text{softmax}(\text{logits})$ 
27: if image unique ( $B = 1$ ) then
28:    $y \leftarrow y.\text{squeeze}(0)$  ▷ (47, )
29: end if
30: return  $y$ 

```

B Entraînement

Algorithme 2 ENTRAÎNEMENT du classifieur

Entrées : Jeu de données EMNIST Balanced, nombre d'épochs $E = 30$, taille de lot $B = 64$, taux d'apprentissage $\eta_0 = 10^{-3}$

Sortie : Modèle f_θ entraîné

```

1: Charger train_dataset avec augmentation (rotation, translation, shear)
2: Charger test_dataset sans augmentation
3: Initialiser le modèle  $f_\theta$  (voir Tableau ??)
4: Initialiser l'optimiseur Adam :  $\eta \leftarrow \eta_0$ 
5: Initialiser l'ordonnanceur : diviser  $\eta$  par 2 tous les 5 épochs
6: Initialiser le critère : entropie croisée avec lissage  $\eta_{\text{smooth}} = 0,1$ 
7: for  $e = 1, \dots, E$  do
8:    $f_\theta.\text{train}()$ 
9:   for chaque mini-lot  $(X, Y) \in \text{train\_loader}$  do
10:     logits  $\leftarrow f_\theta.\text{forward\_training}(X)$ 
11:      $\ell \leftarrow \mathcal{L}_{\text{CE}}(\text{logits}, Y)$   $\triangleright \mathcal{L} = -\sum_c \tilde{y}_c \log p_c$ 
12:     Remise à zéro des gradients
13:     Rétropropagation :  $\nabla_\theta \ell$ 
14:     Mise à jour :  $\theta \leftarrow \theta - \eta \nabla_\theta \ell$ 
15:   end for
16:   Mise à jour du taux d'apprentissage (ordonnanceur par paliers)
17: end for
18: Évaluation :
19:  $f_\theta.\text{eval}()$ 
20: correct  $\leftarrow 0$ , total  $\leftarrow 0$ 
21: for chaque mini-lot  $(X, Y) \in \text{test\_loader}$  do
22:    $\hat{y} \leftarrow \arg \max_c [f_\theta(X)]_c$ 
23:   correct  $\leftarrow \text{correct} + \sum_i \mathbf{1}[\hat{y}_i = Y_i]$ 
24:   total  $\leftarrow \text{total} + |Y|$ 
25: end for
26: Accuracy  $\leftarrow \text{correct}/\text{total}$ 
27: Exporter  $f_\theta$  en TorchScript  $\rightarrow \text{classifiers}/\langle \text{filename} \rangle.\text{pt}$ 
28: return  $f_\theta$ 

```

C Attaque PGD pour le second classifieur

C.1 Attaquant utilisé : Projected Gradient Descent

C.1.1 Explication

La descente de gradient projetée (PGD) est une méthode d'attaque itérative qui perturbe les données d'entrée en suivant le gradient de la fonction de perte d'un modèle afin de maximiser l'erreur, tout en projetant l'entrée perturbée dans un ensemble contraint après chaque étape. Contrairement aux attaques en une seule étape comme FGSM, la PGD applique de multiples petites mises à jour, améliorant les perturbations pour créer des attaques plus robustes.

C.1.2 Opérations fondamentales

Normalisation ℓ_2 . étant donné un tenseur $v \in \mathbb{R}^d$, sa direction unitaire ℓ_2 est

$$\hat{v} = \frac{v}{\max(\|v\|_2, \epsilon_{\text{num}})}, \quad \epsilon_{\text{num}} = 10^{-12}. \quad (11)$$

La constante ϵ_{num} est la plus petite valeur numérique supérieure à 0. Elle permet d'éviter la division par zéro lorsque le gradient est trop proche de 0.

Projection sur la boule ℓ_2 . La projection euclidienne de δ sur $\mathcal{B}_2(0, \varepsilon)$ est

$$\Pi_\varepsilon(\delta) = \delta \cdot \min\left(1, \frac{\varepsilon}{\|\delta\|_2}\right). \quad (12)$$

Si δ est déjà dans la boule ($\|\delta\|_2 \leq \varepsilon$), le facteur vaut 1 et δ est laissé inchangé. Sinon, il est ramené exactement sur le bord de la boule. Cela permet de limiter l'attaque dans les limites données.

Initialisation aléatoire. Pour éviter les optimums locaux, on peut initialiser δ uniformément au hasard à l'intérieur de la boule :

$$\delta^{(0)} = \Pi_\varepsilon(r \cdot \hat{u}), \quad \hat{u} = \frac{u}{\|u\|_2}, \quad u \sim \mathcal{N}(0, I_d), \quad r \sim \mathcal{U}[0, \varepsilon]. \quad (13)$$

La perturbation est ensuite tronquée pour maintenir $x + \delta \in [0, 1]^d$ (fonction clip) :

$$\delta^{(0)} \leftarrow \Pi_\varepsilon(\text{clip}(x + \delta^{(0)}, 0, 1) - x).$$

C.1.3 Calendrier de pas cosinus

à l'itération $t \in \{0, \dots, T-1\}$ avec un pas de base α , un facteur d'amortissement cosinus est appliqué :

$$s_t = \frac{\alpha}{2} \left(1 + \cos\left(\frac{\pi t}{T-1}\right)\right). \quad (14)$$

Ce calendrier démarre à α (exploration agressive) et décroît vers 0 (raffinement fin), ce qui permet une convergence plus stable de l'attaque.

C.1.4 Règle de mise à jour PGD

En partant de $\delta^{(0)}$ (nul ou aléatoire), chaque étape PGD est :

1. **Calcul du gradient :**

$$g^{(t)} = \nabla_{\delta^{(t)}} \mathcal{L}(f_\theta(x + \delta^{(t)}), y). \quad (15)$$

2. **Pas dans la direction du gradient dans ℓ_2 :**

$$\tilde{\delta}^{(t+1)} = \delta^{(t)} + s_t \cdot \hat{g}^{(t)}, \quad \hat{g}^{(t)} = \frac{g^{(t)}}{\|g^{(t)}\|_2}. \quad (16)$$

La normalisation du gradient donne la direction de montée la plus raide ; s_t contrôle la magnitude du pas.

3. **Troncature dans la plage de pixels valide :**

$$\tilde{\delta}^{(t+1)} \leftarrow \text{clip}(x + \tilde{\delta}^{(t+1)}, 0, 1) - x. \quad (17)$$

Cette étape est nécessaire pour que les pixels restent dans la plage de couleur, entre 0 et 1.

4. **Projection sur la boule ℓ_2 :**

$$\delta^{(t+1)} = \Pi_{\epsilon}(\tilde{\delta}^{(t+1)}). \quad (18)$$

En combinant les équations (11)–(18), la mise à jour complète s'écrit :

$$\delta^{(t+1)} = \Pi_{\epsilon}(\text{clip}(x + \delta^{(t)} + s_t \hat{g}^{(t)}, 0, 1) - x). \quad (19)$$

C.2 Algorithme

Algorithme 3 ENTRAÎNEMENT ADVERSARIAL (un epoch)

Entrées : Jeu de données $\mathcal{D}$, modèle f_{θ} , optimiseur, critère $\mathcal{L}_{\eta}$, epoch e , $\epsilon_{\max}$, λ , itérations T_{train}

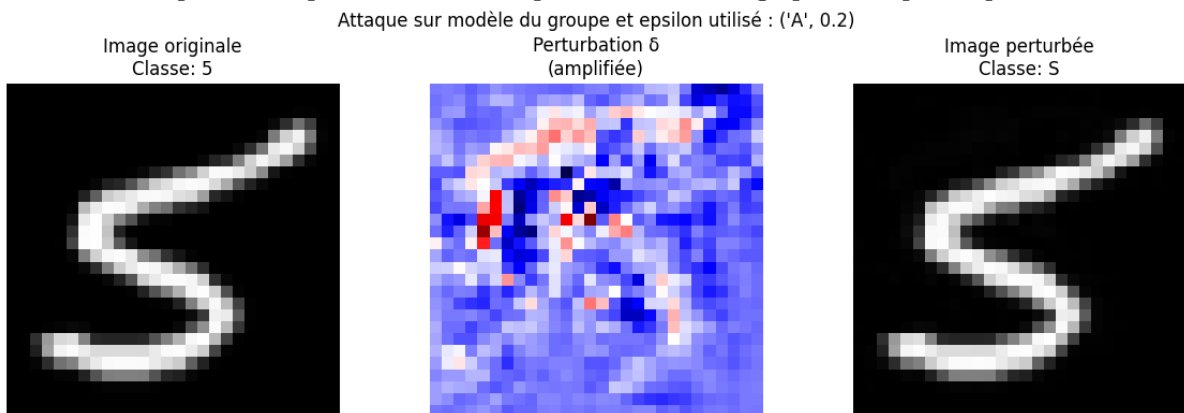
```

1: Calcul du rayon d'entraînement :
2: if  $e < 2$  then
3:    $\epsilon_e \leftarrow 0$ 
4: else
5:    $\epsilon_e \leftarrow \epsilon_{\max} \cdot \min(1, (e - 1)/4)$  ▷ Montée en charge linéaire (équ. 10)
6: end if
7: for chaque mini-lot  $(X, Y) \sim \mathcal{D}$  do
8:    $\ell_{\text{propre}} \leftarrow \mathcal{L}_{\eta}(f_{\theta}(X), Y)$  ▷ Clean loss (équ. 9)
9:   if  $\epsilon_e > 0$  then
10:     $X^* \leftarrow \text{ATTAQUE-PGD-}\ell_2(f_{\theta}, X, Y, \epsilon_e, T_{\text{train}})$ 
11:     $\ell_{\text{adv}} \leftarrow \mathcal{L}_{\eta}(f_{\theta}(X^*), Y)$ 
12:     $\ell \leftarrow (1 - \lambda) \ell_{\text{propre}} + \lambda \ell_{\text{adv}}$  ▷ Clean + adv loss (équ. 8)
13:   else
14:     $\ell \leftarrow \ell_{\text{propre}}$ 
15:   end if
16:   Remise à zéro des gradients; rétropropagation de  $\ell$ ; pas de l'optimiseur
17: end for
18: Mise à jour du scheduler de taux d'apprentissage

```

D Exemples d'attaques Deepfool

Voici un exemple d'attaque réussie de Deepfool sur une image pour un petit epsilon.



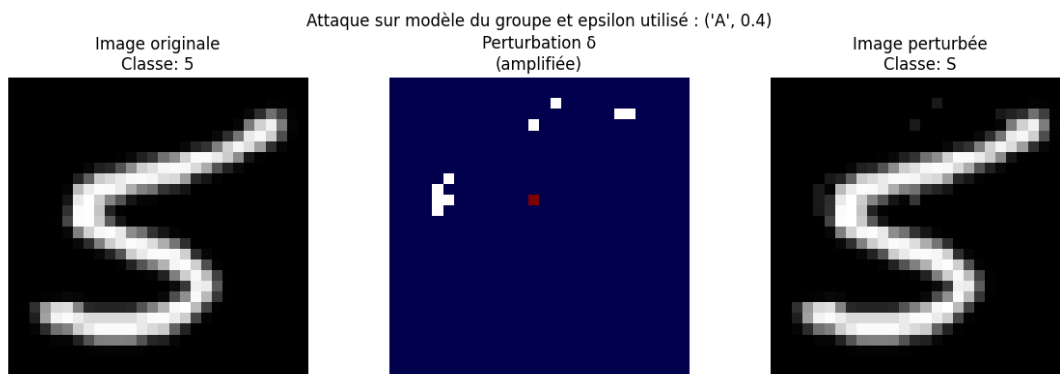
E Autres pistes envisagées

Fast Gradient Signed Method

Nous avons implémenté un attaquant sur base de cette méthode, mais nous nous sommes rendu compte qu'il n'était pas très performant. Cette méthode bien que rapide, ne regarde en fait le gradient qu'une seule fois. Ce qui la rend bien moins performante que la méthode du PGD, qui est en fait sa version itérative. Celle-ci est bien plus puissante.

JSMA Method

C'est un algorithme qui a pour but de modifier un pixel cible à chaque itération pour augmenter les chances de succès de l'attaque. Pour se faire, on utilise la Jacobienne du modèle, on construit une carte de saillance et on choisit les pixels qui augmentent une classe cible tout en diminuant la classe actuelle. C'est une perturbation très localisée dont un exemple est disponible plus bas. En raison des calculs à réaliser et de la recherche des pixels critiques, cette méthode prend beaucoup de temps à s'exécuter et ne donne pas de bien meilleurs résultats que PGD.



(a) Exemple d'attaque JSMA

FIGURE 15

F Exemples d'utilisation du programme de dessin interactif

Cette annexe regroupe plusieurs sorties générées par notre programme de dessin interactif : exemples de dessins, résultats de classification, histogrammes et estimation du seuil adversarial minimal.

F.1 Programme avec epsilon fixé

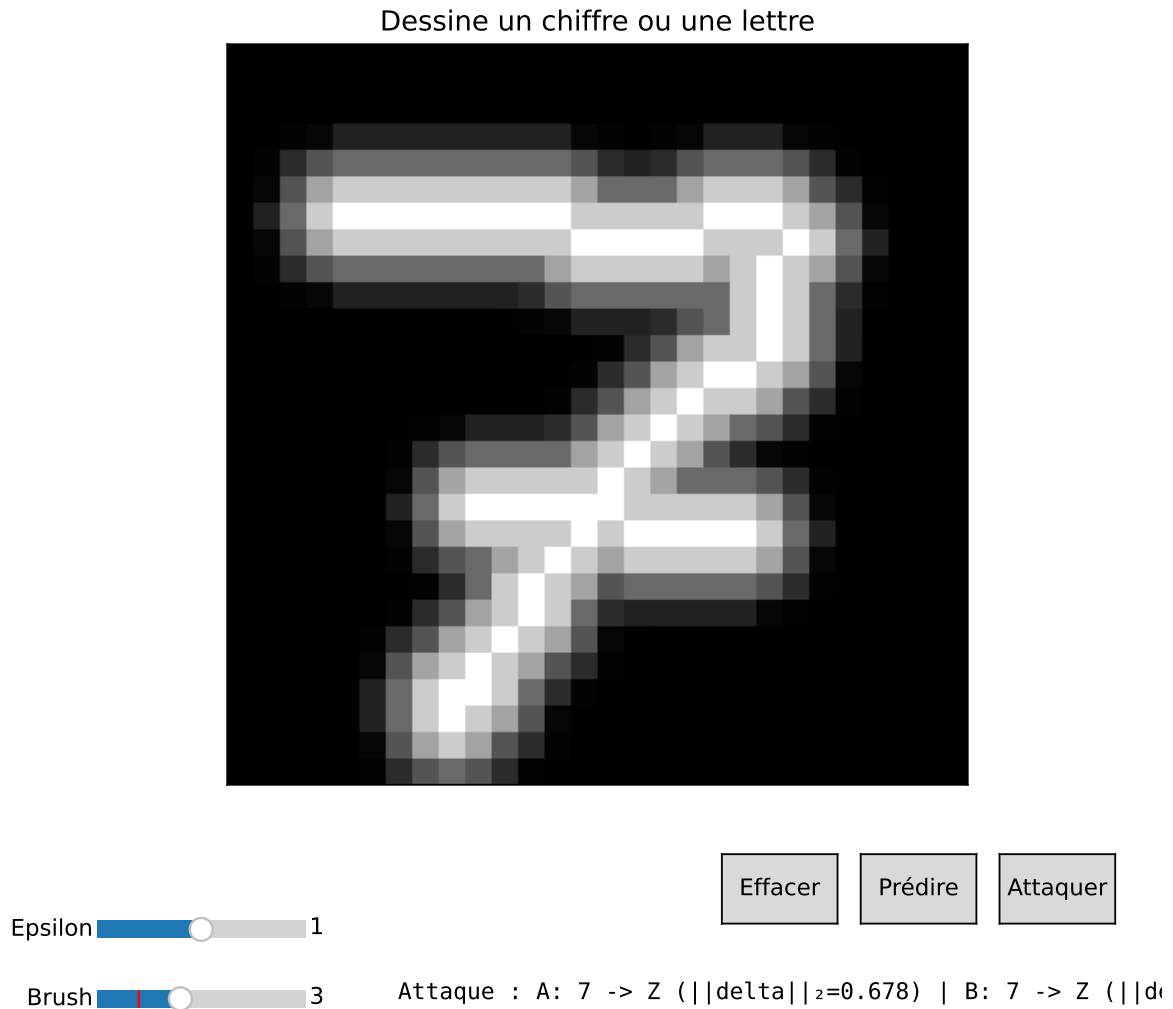


FIGURE 16 – Programme permettant de dessiner pour tester différents classifieurs et attaquants avec un ε donné

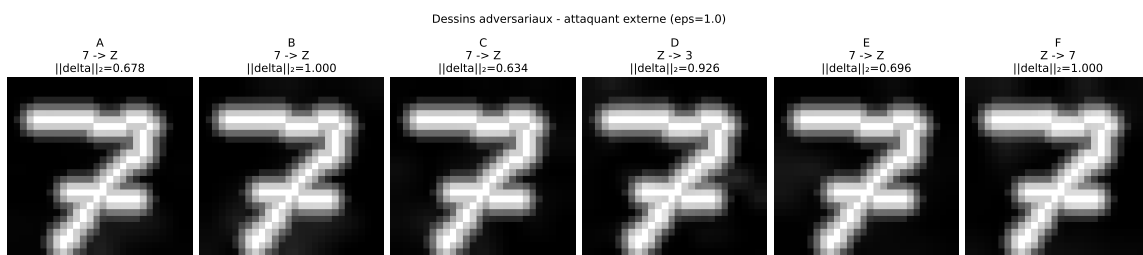


FIGURE 17 – Résultat sous perturbation ε fixée

F.2 Programme permettant de trouver le plus petit epsilon fonctionnel

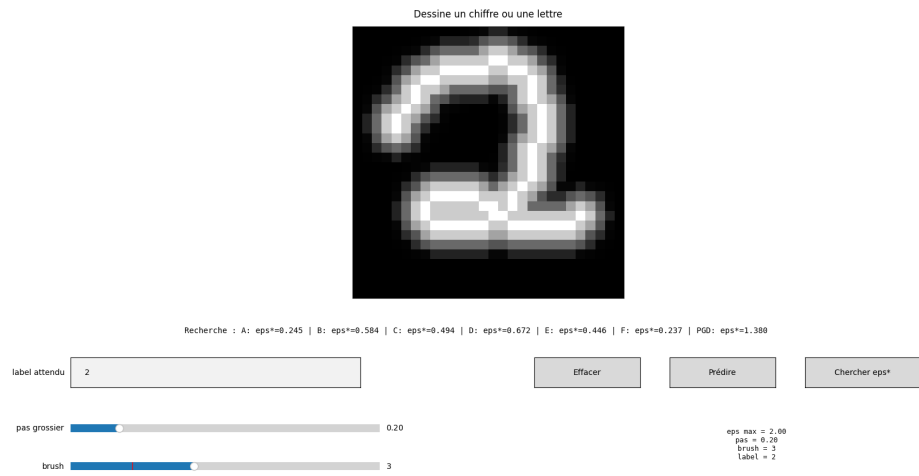


FIGURE 18 – Programme permettant de trouver le plus petit ϵ

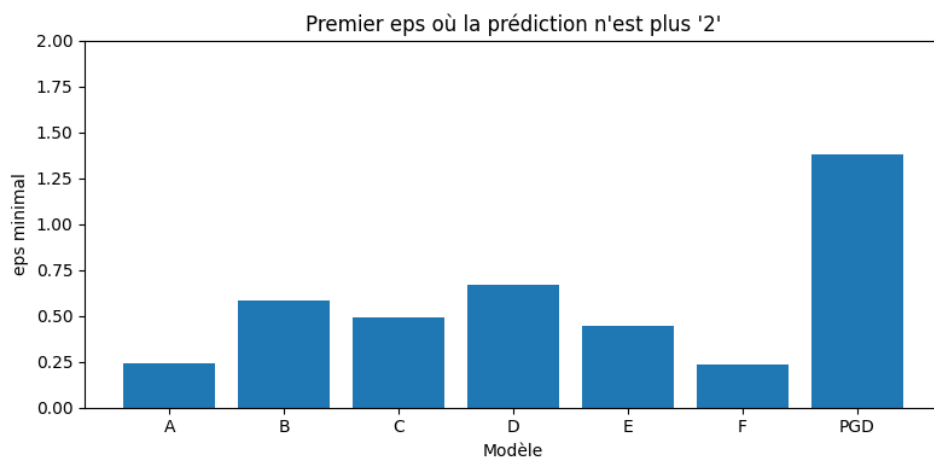


FIGURE 19 – Histogramme des valeurs minimales de perturbation trouvées.

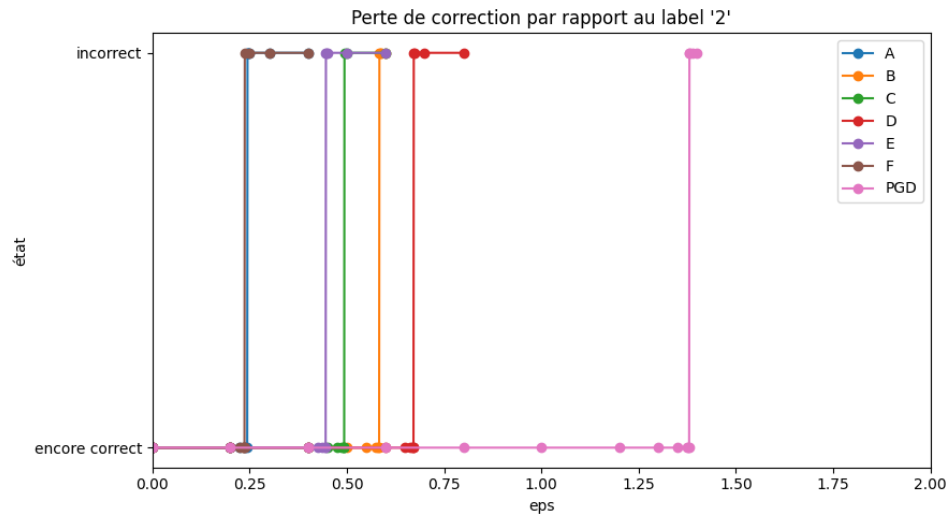


FIGURE 20 – Évolution de la prédiction en fonction du seuil de perturbation.

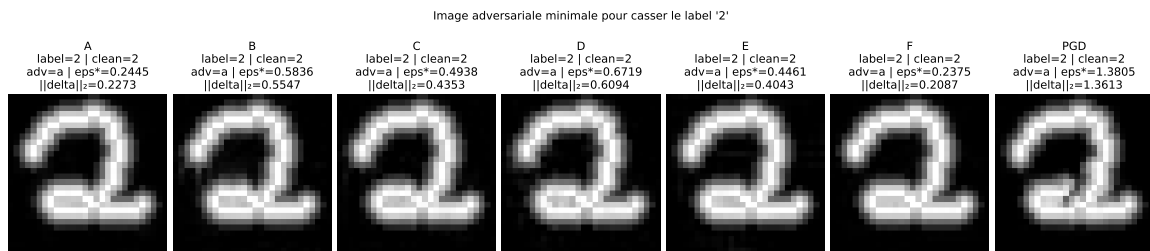


FIGURE 21 – Résultat avec le plus petit ε efficace